we're using the `?:` syntax to mark it as optional.)

Within the template literal for the component, we can use the props by adding functions to be interpolated into the text. We set the `background-color` to `background-color: ${(props) => stateColor(props.status)};`, which sets the color based on the `stateColor` function defined at the top of the file. The `stateColor` is the same function as before except that we've moved it out of the `Seat` component, and we're now returning CSS colors directly, rather than Tailwind class names. We could use Twin, referenced earlier, to give us access to Tailwind class names here, but it's not necessary for this example.

It's a little annoying that you need to define the interpolation as a function, `${(props) => stateColor(props.status)}`, rather than directly, `${stateColor(props.status)}`, but it seems like you need the functional version to have the `props` variable be visible inside the interpolated expression.

We use the props again, this time to help define a hover special status. One nice feature of styled-components is that it has full support for pseudo-elements and nested elements, so we can easily add a hover state. In this case, the hover background color is `lightblue` if the state is `unsold`, otherwise, the hover background is the same as the existing background color.

We also added a static `transition` so that the background colors fade in and out.

Finally, we have to call our new `ButtonSquare` component, which we do as any other React component:

chapter_05/07/app/javascript/components/seat.tsx

```
  return (
    <td>
      <ButtonSquare status={status} onClick={changeState}>
        {seatNumber + 1}
      </ButtonSquare>
    </td>
  )
```

We pass the props as we would with a regular React component, including the